

使用 Rust 语言 Embassy 嵌入式框架实现 STM32WL LoRa 数据传输

Jan 23, 2023 • andelf • Tags: embedded, stm32, embassy, rust, lora

Commit	Date	Commit Message	
69544ab	2024-06-17 20:43:43 +0800	fix: minor update	View this version
433eeb5	2023-02-23 20:05:56 +0800	minor reformat	View this version
83f2206	2023-01-24 10:16:25 +0800	minor update for post	View this version
a9baa89	2023-01-24 00:59:59 +0800	save new posts	View this version



- 介绍
 - 开发板介绍
 - 预备知识
 - 软件环境准备
- 从 Blinky 开始 Embassy 应用开发
 - 创建项目 - 初始化 Rust 嵌入式项目模板
 - Blinky 点灯 - 初识 Rust Embassy
 - UART 打印 - 时钟和外设初始化
 - I2C 访问 BMP280
 - 硬件准备
 - BMP280 访问
 - 代码实现
- LoRa 传感器数据传输
 - 硬件准备
 - 射频开关 RadioSwitch
 - LoRa 数据报文定义
 - SubGhz 初始化

- [SubGhz 发送端](#)
 - [SubGhz 接收端](#)
 - [运行结果](#)
 - [总结](#)
 - [参考资料](#)
-

2024-06 补充: 相关内容已经过期, 且 [lora-rust](#) 项目长达 1 年多没有时间删去 CN470 支持后没有再加回来. 本文仅作为历史记录.

之前参与了 [eet-china.com](#) 的开发板测评活动, 申请的板子是 [STM32WLE5 易智联 Lora 评估板\(LM401-Pro-Kit\)](#), 正好 Rust Embassy 框架对 STM32WL 系列及其 SubGhz 有不错的支持, 所以打算用这套技术栈进行开发尝试.

本文主要介绍如何使用 Rust 语言的 Embassy 嵌入式框架实现 STM32WL LoRa 数据传输. 过年回老家, 随身带的东西不多, 只有一个迷你 BMP280 (大气压温度)传感器模块, 所以本文使用 BMP280 传感器数据作为例子.

门槛率高, 还是从点灯开始搞起.

最终相关代码位于 [Github: andelf/lm401-pro-kit](#).

介绍

快递于 [[2023-01-08]] 收到, 里面的评估板, 天线, 数据线均是两份, 方便开发使用.

开发板介绍

LM401-Pro-Kit 是基于 STM32WLE5CBU6 的 Lora 评估板. 支持 SubGHz 无线传输. LM401 模组内嵌高性能 MCU 芯片 STM32WLE5CBU6, 芯片内部集成了 SX1262. 开发板板载 ST-Link(上传下载程序, UART 转 USB). ST-Link 通过跳线帽和模块核心部分连接, 方便单独供电使用模块. 开发板提供了若干 LED 状态灯, 复位按钮和一个用户按钮.

日常屯的(吃灰)板子也有大几十上百了, 拿到新板子, 需要查资料, 看手册, 电路图, 读例程, 找到一些核心信息, 其中一些信息可能需要读例程的 C 代码库才能获得, 这里列出整理的部分:

- MCU: STM32WLE5CBU6
 - 架构: Cortex-M4
 - 主频: 48MHz, 通过 **MSI** 提供
 - FLASH 128K, RAM 48K
 - 核心外设: SX1262 via SPI3
- LM401: CN470-510MHZ
- 板载
 - ST-Link 下载器
 - 用户按钮 PA0
 - LED blue PB5, green PB4, yellow PB3

- 射频开关:
 - FE_CTRL1 PB0
 - FE_CTRL2 PA15, LM401 未使用
 - FE_CTRL3 PA8
- UART RX/TX
 - PA2 TXD, USART2_TX, LPUART1_TX
 - PA3 RXD, USART2_RX, LPUART1_RX

预备知识

- 对 Rust 的基础了解
- 对 STM32 的基础了解

使用 Rust 嵌入式开发大概大概有如下几层(只是粗略分类, 实际项目使用中, 可能会混合使用):

- 直接使用 PAC 库操作寄存器, PAC 库通过 `svd2rust` 工具从 `.svd` 文件生成
- 使用 HAL 库, 例如 `stm32f4xx-hal`, `stm32l0xx-hal`, `stm32wlxx-hal` 等, 融合 `embedded-hal` 生态
- 使用 Rust 嵌入式框架, 例如 `embassy`

Embassy 框架是基于 Rust 语言的嵌入式异步框架. 考虑到相关框架还在开发中, 本文的代码仓库使用的是最新的 embassy master 分支. Commit hash 为

`f98ba4ebac192e81c46933c0dc1dfb2d907cd532`, 通过 `Cargo.toml` 中设置依赖 `path` 的方式引入. 其他可选方案还可有 `git submodule` 或直接 `git` 依赖远程版本等.

绕开 C HAL/BSP 库开发, 是需要踩不少坑的, 例如, RCC 时钟初始化, 需要查阅 BSP 代码才能确认, 48MHz 主时钟通过 MSI range11 获得, 而 embassy 对应 MCU 的示例代码使用的是 HSE, 这些都给 Rust 嵌入式开发带来一定的门槛.

软件环境准备

安装 Rust 工具链, 本文使用 `rustup nightly`. 请参考 <https://rustup.rs/>.

安装 Rust `thumbv7em-none-eabi` target, 对应 Cortex-M4:

```
rustup target add thumbv7em-none-eabi
```

安装 Rust 嵌入式开发烧录/运行工具 `probe-run`, 也可以使用 OpenOCD 或其他烧录工具:

```
> cargo install probe-run
...(install log)
> probe-run --list-chips | grep STM32WL
STM32WL Series
  STM32WLE5J8Ix
  STM32WLE5JB1x
```

```
STM32WLE5JCIx
STM32WL55JCIx
```

检查发现支持列表没有 STM32WLE5CBU6, 不过可以拿 STM32WLE5JCIx 替代, 问题不大.

安装任意串口调试工具, 这里我使用 `picocom`. 其他可以使用的替代有 `PuTTY`, `Teraterm` 等等.

通过 USB 数据线连接开发板, 通过 `picocom` 连接串口, 通过 `probe-run` 烧录程序.

测试连接

```
> lsusb
Bus 001 Device 008: ID 0483:374b STMicroelectronics STM32 STLink Serial
```

从 Blinky 开始 Embassy 应用开发

考虑到从初识 Rust 嵌入式开发直接跨越到 LoRa 无线传输门槛较高, 我们从简单的点灯例子开始:

创建项目 - 初始化 Rust 嵌入式项目模板

我们直接依赖 embassy 的 master 分支进行开发, 为方便调试, 直接 clone 到本地用相对路径引入依赖:

```
git clone git@github.com:embassy-rs/embassy.git
# or
git clone https://github.com/embassy-rs/embassy.git

# 在同层目录直接创建我们的项目, 起板子名就可以. 相当于一个 BSP 模板可以扩充

cargo new --lib lm401-pro-kit

# 进入项目目录, 以下命令均在此执行
cd lm401-pro-kit
```

Rust 嵌入式项目的初始设置需要请参考项目代码

- `.cargo/config.toml`
 - 设置编译器 target 到 `thumbv7em-none-eabi`
 - 设置 `cargo run` 的执行方式为调用 `probe-run ...`
 - ```
[target.'cfg(all(target_arch = "arm", target_os = "none"))']
runner = "probe-run --chip STM32WLE5JCIx"

[build]
target = "thumbv7em-none-eabi"
```

- `build.rs`
  - 设置 `link.x` / `memory.x` 链接过程中所用配置, 编译过程中由 `embassy` 自动按照芯片选择生成
  - 添加 `defmt` 链接参数支持
- `Cargo.toml`
  - 添加 `embassy` 相关依赖, 并通过 `features` 设置相关参数
  - 添加项目依赖, `defmt`, `cortex-m` 相关等
  - 设置编译参数 `opt-level = "z"`, 最小化编译二进制大小
  - ```
# part of Cargo.toml
[dependencies]
# ...
embassy-stm32 = { version = "0.1.0", path = "../embassy/embassy-s
    "nightly",
    "defmt",
    "stm32wle5cb",
    "time-driver-any",
    "memory-x",
    "subghz",
    "unstable-pac",
    "exti",
] }
# ...
[profile.dev]
opt-level = "z" # Optimize for size.

[profile.release]
lto = true
opt-level = "z" # Optimize for size.
```
 - `defmt` 是一个非常好用的 Rust 嵌入式调试打印, 对 STM32(ST-Link) 有很好的支持.
 - `stm32wle5cb` 用于选择 STM32WLE5CBU6 的芯片配置, `subghz` 用于选择 SubGHz 驱动.
 - `memory-x` 自动生成链接所需的 `memory.x` 文件(FLASH, SRAM 的大小和内存位置).
- 为避免编译报错, 还需要清空 `src/lib.rs` 项目初始文件, 用 `#![no_std]` 替代

几乎所有的 Rust 嵌入式项目都是 `no_std` 的, 这意味着无法简单地使用所有带内存分配类型. 本例中, 我们使用 `heapless` crate 中提供的栈分配类型来替代 `String`.

注意到, 创建项目时候使用了 `cargo new --lib`, 相当于我们创建的是一个 library 项目. 这不需要担心, `cargo run` 会自动识别 `src/bin/xxx.rs` 为“可执行”二进制目标. 通过 `cargo run --bin xxx` 即可运行对应程序. 也可以通过 `examples/xxx.rs` 的方法管理多个可执行二进制目标.

Blinky 点灯 - 初识 Rust Embassy

我们先通过一个最简单的闪灯例子来熟悉 Rust Embassy 的使用. 创建

`src/bin/blinky.rs`.

```
// blinky.rs
#![no_std]
#![no_main]
#![feature(type_alias_impl_trait)]

use defmt::*;
use embassy_executor::Spawner;
use embassy_stm32::gpio::{Level, Output, Speed};
use embassy_time::{Duration, Timer};
use {defmt_rtt as _, panic_probe as _};

#[embassy_executor::main]
async fn main(_spawner: Spawner) {
    let p = embassy_stm32::init(Default::default());
    info!("Hello World!");

    let mut led = Output::new(p.PB4, Level::High, Speed::Low);

    loop {
        info!("high");
        led.set_high();
        Timer::after(Duration::from_millis(1000)).await;

        info!("low");
        led.set_low();
        Timer::after(Duration::from_millis(1000)).await;
    }
}
```

`#![no_main]` 用于告诉 Rust 编译器, 我们不使用 Rust 提供的 `main` 函数做程序入口. `#[embassy_executor::main]` 是一个宏, 用于包装 `async fn main()` 函数, 由 `embassy-executor` 提供了一个 futures runtime, 所以可以使用 `async` 和 `await` 语法. 底层实现中, `.await` 通过 STM32 的 WFE/SEV 等待指令和中断唤醒指令实现, 实现了程序逻辑在等待时候的低功耗. `Spawner` 还可以用来启动其他 `async fn` 函数, 实现了多任务的功能.

`#![feature(type_alias_impl_trait)]` 在 `embassy` 中被广泛使用, 需要开启. Embassy 中经常能看到形如 `irq: impl Peripheral<P = T::Interrupt> + 'd` 的类型签名.

`let p = embassy_stm32::init(Default::default());` 直接初始化了所有的外设, 并返回一个 `Peripherals` 对象. 通过 Rust 的 `move` 语义保证不同外设使用之间不会出现竞争.

`let mut led = Output::new(p.PB4, Level::High, Speed::Low);` 创建了一个 `Output` 对象, 用于控制 PB4 引脚. `Output` 对象是一个 `Pin` 的 trait, 通过 `set_high` 和 `set_low` 方法可以控制引脚电平. 这里会自动完成对 GPIOB PB4 的所有初始化和设置, 包括外设时钟使能, 状态设置等.

`info!`, `warn!` 等都是 `defmt` 的宏, 用于通过 ST-Link 提供的 Debug 通道打印调试信息. 强烈推荐使用, 否则嵌入式开发中, 只能用串口打印信息.

`Timer::after(Duration::from_millis(1000)).await` 是一个异步等待 1 秒的方法, 通过 `embassy-time` crate 实现. 在 `Cargo.toml` 中的 `time-driver-any` feature 选择了任意可用 timer 实现, 默认是 TIM2, 由 `embassy-stm32` 提供给 `embassy-time`.

确保板子连接正常, 直接运行:

```
> cargo run --bin blinky
    Finished dev [optimized + debuginfo] target(s) in 0.32s
    Running `probe-run --chip STM32WLE5JCIx target/thumbv7em-none-eabi/`
(HOST) INFO flashing program (14 pages / 14.00 KiB)
(HOST) INFO success!

0.000000 DEBUG rcc: Clocks { sys: Hertz(4000000), apb1: Hertz(4000000),
└ embassy_stm32::rcc::set_freqs @ ./embassy/embassy-stm32/src/fmt.rs:12
0.000113 INFO Hello World!
└ blinky:_____embassy_main_task::{async_fn#0} @ src/bin/blink.rs:14
0.000552 INFO high
└ blinky:_____embassy_main_task::{async_fn#0} @ src/bin/blink.rs:19
1.001157 INFO low
└ blinky:_____embassy_main_task::{async_fn#0} @ src/bin/blink.rs:23
2.001811 INFO high
```

- 二进制编译成功后, 由 `probe-run` 烧录到 MCU 并执行, 持续获取 `defmt` 打印信息
- `rcc: Clocks` 调试时钟信息由 `embassy-stm32` 库 `embassy_stm32::rcc::set_freqs` 打印
- 所有 `defmt` 打印内容在 `cargo run` dev 模式下均附加了代码行, 非常方便
- `defmt` 打印内容均带有时间戳, 该时间戳由 STM32 SYSTICK 提供(所以如果使用了 SYSTICK, 有可能导致时间戳异常)
- 最终的 `main` 函数显示为 `blink.rs:_____embassy_main_task::{async_fn#0}`, 由 `#[embassy_executor::main]` 宏生成

UART 打印 - 时钟和外设初始化

`defmt` 固然方便, 但很多时候依然需要用到 UART, 通过串口获取调试信息或收集数据. LM401-Pro-Kit 正好通过 ST-Link 提供了到 USART2 的访问.

Blinky 例子中, 由 defmt 调试信息可知, 我们使用的系统时钟只有 4MHz, 但 STM32WL 的最大时钟频率是 48MHz. 所以需要通过初始化 `init()` 方法设置时钟参数:

```
// sys clk init, with LSI support
let mut config = embassy_stm32::Config::default();
config.rcc.enable_lsi = true;
config.rcc.mux = embassy_stm32::rcc::ClockSrc::MSI(embassy_stm32::rcc::Msi::new());
let p = embassy_stm32::init(config);
```

Embassy UART 使用非常简单, 可以单独用 UartTx/UartRx 只初始发送/接收部分. 这里是一个发送 Hello world 和 MCU 内部 “时间” 的简单示例:

```
// USART2 tx
use embassy_stm32::dma::NoDma;
use embassy_stm32::uart::UartTx;
use embassy_time::Instant;
use heapless::String;

// Default: 115200 8N1
let mut uart = UartTx::new(p.USART2, p.PA2, NoDma, Default::default());

let mut msg: String<64> = String::new();
let i = Instant::now();
core::write!(msg, "Hello world, device time: {}\r\n", i.as_millis()).unwrap();
uart.blocking_write(msg.as_bytes()).unwrap();
msg.clear();
```

`UartTx` 初始化时需要传入 `USART2`, `PA2`, 分别对应 USART2 外设和 TX 引脚, DMA 通道是可选的. 默认串口参数是 115200 8N1. 外设初始化会自动处理对应引脚的 AF 设置.

串口打印需要字符串拼接格式化, 由于 `no_std`, 标准库的 `String` 类型不可用, 这里使用 `heapless::String`, 初始化时候需要指定分配大小. `core::write!` 即标准库中的 `write!`, `core::` 前缀是为了避免和 `defmt::write!` 名字冲突.

完整代码请参考 [代码仓库](#).

执行代码确认, 可以看到系统时钟被正确设置为 48MHz.

```
> cargo run --bin uart
0.000000 DEBUG rcc: Clocks { sys: Hertz(48000000), apb1: Hertz(48000000)
└─ embassy_stm32::rcc::set_freqs @ /Users/mono/Elec/embassy/embassy-stm32
0.000011 INFO Hello World!
└─ uart::__embassy_main_task::{async_fn#0} @ src/bin/uart.rs:21
0.000064 INFO tick
```

在另一命令行打开串口监视工具, 查看串口输出:


```
> picocom -b 115200 /dev/tty.usbmodem11103
Hello world, device time: 1000
Hello world, device time: 3002
Hello world, device time: 5005
Hello world, device time: 7008
Hello world, device time: 9011
Hello world, device time: 11013
....
```

I2C 访问 BMP280

硬件准备

- BMP280 传感器模块 1 个
- 杜邦线若干根, 用于连接传感器模块和开发板

BMP280 是来自 Bosch 的气压传感器, 通过 I2C 接口读取气压和温度数据, 所以需要在板子上找到未被占用的 I2C SCL/SDA 引脚资源, 通过查阅芯片手册, 最后选择了空闲的 I2C2, SCL pin PA12, SDA pin PA11. 开发板上一排跳线帽正好提供了 VCC, GND.

接线:

```
+-----+          VCC GND
| BMP280 |          |   |
|   VCC  >-----+   |
|   GND  >-----+
| [.] SCL >----->PA12
|   SDA  >----->PA11
|         |          (LM401-Pro-Kit)
+-----+
```

BMP280 访问

Rust Embassy 完美兼容 `embedded-hal` 相关生态, 相关外设类型均支持对应的 `embedded-hal` trait,

考虑到 BMP280 的使用略微复杂, 需要初始化, 读取校准数据, 测量后还需要通过校准数据计算最终测量结果. 所以 BMP280 直接寻找对应驱动即可. 但 Rust 嵌入式生态有个问题, 弃坑项目太多. 寻找第三方依赖时候需要注意阅读代码, 查看依赖版本, 必要时更新.

这么说, 其实是之前我有个弃坑项目里面有个 BME280 驱动库, BME280 和 BMP280 基本兼容, 只是多了湿度测量. 驱动代码使用 `embedded-hal` 提供的 trait 类型访问设备, 完成传感器初始化和测量. 稍微改了改, 直接 Copy [embedded-drivers: bme280.rs](#) 到项目 `src/` 下使用即可.

修改 `src/lib.rs` 增加:

```
pub mod bme280;
```

代码实现

创建 BMP280 传感器项目 `src/bin/i2c-bmp280.rs`. 完整代码请参考 [代码仓库](#), 以下只选择关键部分介绍.

```
// BMP280 init
use embassy_stm32::i2c::I2c;
use embassy_stm32::interrupt;
use embassy_stm32::time::Hertz;
use embassy_time::Delay;
use lm401_pro_kit::bme280::BME280;

let irq = interrupt::take!(I2C2_EV);
let i2c = I2c::new(
    p.I2C2,
    p.PA12,
    p.PA11,
    irq,
    NoDma,
    NoDma,
    Hertz(100_000),
    Default::default(),
);

let mut delay = Delay;

let mut bmp280 = BME280::new_primary(i2c);
unwrap!(bmp280.init(&mut delay));
```

Embassy 中访问设备时, 一般会需要中断, 虽然理论上阻塞访问外设时不需要中断. 但是为了保证接口的一致性, 一般都会要求提供中断参数. `interrupt::take!` 用于获取对应中断对象.

`BME280::new_primary` 直接使用设备主地址 `0x76` 访问 I2C 总线上的 BMP280.

初始化设备时候由于需要软复位, 需要传递 `Delay` 对象, 用于延时(`delay_ms`). 默认的 `embassy_time::Delay` 使用循环比较 "设备当前时间" 的方法实现.

`unwrap!` 宏由 `defmt` 提供, 等价于 `.unwrap()` 调用, 但是会在 panic 时候通过 `defmt` 打印信息.

完成设备初始化后, 可以访问传感器信息:

```
let raw = unwrap!(bmp280.measure(&mut delay));
info!("BMP280: {:?}", raw);
```

传感器执行测量时候, 按照手册, 依然需要延时, 所以也同样需要传递 `Delay` 对象.

`BME280::measure` 方法返回 `Measurements` 类型, 为了方便调试使用, 用 `derive macro` 增加了 `defmt` 支持, 可以直接做格式化参数:

```
#[derive(Debug, defmt::Format)]
pub struct Measurements {
    /// temperature in degrees celsius
    pub temperature: f32,
    /// pressure in pascals
    pub pressure: f32,
    /// percent relative humidity (`0` with BMP280)
    pub humidity: f32,
}
```

执行代码:

```
> cargo run --bin i2c-bmp280
0.000011 INFO I2C BMP280 demo!
└─ i2c_bmp280::____embassy_main_task::{async_fn#0} @ src/bin/i2c-bmp280.
0.009314 INFO measure tick
└─ i2c_bmp280::____embassy_main_task::{async_fn#0} @ src/bin/i2c-bmp280.
0.051652 INFO BMP280: Measurements { temperature: 23.689554, pressure:
└─ i2c_bmp280::____embassy_main_task::{async_fn#0} @ src/bin/i2c-bmp280.
```

temperature: 23.689554, pressure: 88391.13 传感器数据正常.

LoRa 传感器数据传输

LoRa 是一种无线传输协议, 适合长距离(km), 少量数据传输. 尤其适合传感器数据. 因为手头没有 LoRaWAN 基站, 所以暂时没法测试 LoRaWAN. 这里使用 LoRa 调制模式点对点传输 BMP280 传感器数据.

详细实现请参考 [代码仓库](#) 里的 `src/bin/subghz-bmp280-tx.rs` 和 `src/bin/subghz-bmp280-rx.rs`.

硬件准备

LM401-Pro-Kit x2, 天线, 数据线.

其中一个开发板作为传感器采集端, 按照上一示例链接到 BMP280 传感器模块, 另一个作为接收端, 两个开发板之间通过 LoRa 无线传输数据. 接收端通过 UART 与电脑连接, 通过串口调试工具查看传感器数据.(实际上也可以直接通过 ST-Link + `defmt` 获取数据)

射频开关 RadioSwitch

使用开发板射频功能, 需要处理射频开关逻辑. 相关逻辑从 BSP C 代码获得. 可以直接作为 BSP 的工具类型, 写入到 `src/lib.rs` 中:

```
use embassy_stm32::{
    gpio::{AnyPin, Level, Output, Pin, Speed},
    peripherals::{PA15, PA8, PB0},
};

pub struct RadioSwitch<'a> {
    ctrl1: Output<'a, AnyPin>,
    ctrl2: Output<'a, AnyPin>,
    ctrl3: Output<'a, AnyPin>,
}

impl<'a> RadioSwitch<'a> {
    pub fn new_from_pins(ctrl1: PB0, ctrl2: PA15, ctrl3: PA8) -> Self {
        Self {
            ctrl1: Output::new(ctrl1.degrade(), Level::Low, Speed::VeryHigh),
            ctrl2: Output::new(ctrl2.degrade(), Level::Low, Speed::VeryHigh),
            ctrl3: Output::new(ctrl3.degrade(), Level::Low, Speed::VeryHigh),
        }
    }

    pub fn new(
        ctrl1: Output<'a, AnyPin>,
        ctrl2: Output<'a, AnyPin>,
        ctrl3: Output<'a, AnyPin>,
    ) -> Self {
        Self {
            ctrl1,
            ctrl2,
            ctrl3,
        }
    }

    pub fn set_off(&mut self) {
        self.ctrl3.set_low();
        self.ctrl1.set_low();
        self.ctrl2.set_low();
    }
}

impl<'a> embassy_lora::stm32wl::RadioSwitch for RadioSwitch<'a> {
    fn set_rx(&mut self) {
        self.ctrl3.set_low();
        self.ctrl1.set_high();
        self.ctrl2.set_low();
    }

    fn set_tx(&mut self) {
```

```

        self.ctrl3.set_high();
        self.ctrl1.set_low();
        self.ctrl2.set_low();
    }
}

```

非常简单的 GPIO 操作, GPIO 的强类型 `PAn / PBn` 可以通过 `.degrade()` 方法转换为 `AnyPin` 类型, 方便使用.

```
let mut rfs = lm401_pro_kit::RadioSwitch::new_from_pins(p.PB0, p.PA15, p
```

LoRa 数据报文定义

为简单展示, 传感器节点只负责发送, 接受节点只接受 LoRa 报文, 不回传 ACK 信号.

报文格式为 24 字节:

头	设备地址	设备时间戳	温度	大气压	checksum
b"MM"	u32	u64	f32	f32	u16

其中设备地址使用 STM32 系列的 chip id 实现, 保证一定的唯一性:

```

// Device ID in STM32L4/STM32WL microcontrollers
pub fn chip_id() -> [u32; 3] {
    unsafe {
        [
            core::ptr::read_volatile(0x1FFF7590 as *const u32),
            core::ptr::read_volatile(0x1FFF7594 as *const u32),
            core::ptr::read_volatile(0x1FFF7598 as *const u32),
        ]
    }
}

let chip_id = chip_id();
let dev_addr = chip_id[0] ^ chip_id[1] ^ chip_id[2];

```

设备时间戳直接读取 `Instant::now()` 并转为 `millis`. 保证每个数据报文的差异性.

`checksum` 校验和字段通过计算 `[2..22]` 所有字节之和得到. 所有数据字段均按照大端序列化(BigEndian).

SubGhz 初始化

LM401 的射频功能由 STM32WLE5 内置的 SX1262 提供, 设备内部通过 SPI3(SUBGHZSPI) 访问. SX1262 初始化需要较多参数, 且发送端接收端若干参数需要一致.

这里选用 490.500MHz, LoRa SF7, 4/5 编码率, 125kHz 带宽, 24 字节数据长度. 接收端和发送端设置一致.

参数定义:

```
use embassy_stm32::subghz::*;

const DATA_LEN: u8 = 24_u8;
const PREAMBLE_LEN: u16 = 0x8 * 4;

const RF_FREQ: RfFreq = RfFreq::from_frequency(490_500_000);

const TX_BUF_OFFSET: u8 = 128;
const RX_BUF_OFFSET: u8 = 0;
const LORA_PACKET_PARAMS: LoRaPacketParams = LoRaPacketParams::new()
    .set_crc_en(true)
    .set_preamble_len(PREAMBLE_LEN)
    .set_payload_len(DATA_LEN)
    .set_invert_iq(false)
    .set_header_type(HeaderType::Fixed);

// SF7, Bandwidth 125 kHz, 4/5 coding rate, low data rate optimization
const LORA_MOD_PARAMS: LoRaModParams = LoRaModParams::new()
    .set_bw(LoRaBandwidth::Bw125)
    .set_cr(CodingRate::Cr45)
    .set_ldro_en(true)
    .set_sf(SpreadingFactor::Sf7);

// see table 35 "PA optimal setting and operating modes"
const PA_CONFIG: PaConfig = PaConfig::new()
    .set_pa_duty_cycle(0x4)
    .set_hp_max(0x7)
    .set_pa(PaSel::Hp);

const TX_PARAMS: TxParams = TxParams::new()
    .set_power(0x16) // +22dB
    .set_ramp_time(RampTime::Micros200);
```

设备初始化, 部分内容从 BSP C 代码转换得到:

```
let mut radio = SubGhz::new(p.SUBGHZSPI, NoDma, NoDma);

// from demo code: Radio_SMPS_Set
unwrap!(radio.set_smpps_clock_det_en(true));
unwrap!(radio.set_smpps_drv(SmpsDrv::Milli40));
```

```

unwrap!(radio.set_standby(StandbyClk::));

// in X0 mode, set internal capacitor (from 0x00 to 0x2F starting 11.2pF)
unwrap!(radio.set_hse_in_trim(HseTrim::from_raw(0x20)));
unwrap!(radio.set_hse_out_trim(HseTrim::from_raw(0x20)));

unwrap!(radio.set_regulator_mode(RegMode::Smps)); // Use DCDC

unwrap!(radio.set_buffer_base_address(TX_BUF_OFFSET, RX_BUF_OFFSET));

unwrap!(radio.set_pa_config(&PA_CONFIG));
unwrap!(radio.set_pa_ocp(Ocp::Max60m)); // current max
unwrap!(radio.set_tx_params(&TX_PARAMS));

unwrap!(radio.set_packet_type(PacketType::LoRa));
unwrap!(radio.set_lora_sync_word(LoRaSyncWord::Public));
unwrap!(radio.set_lora_mod_params(&LORA_MOD_PARAMS));
unwrap!(radio.set_lora_packet_params(&LORA_PACKET_PARAMS));
unwrap!(radio.calibrate_image(CalibrateImage::ISM_470_510));
unwrap!(radio.set_rf_frequency(&RF_FREQ));

```

中断信号量处理, 由于发送接收循环需要涉及到中断处理, 这里直接用 `Signal` 类型的信号量处理中断:

```

use embassy_sync::blocking_mutex::raw::CriticalSectionRawMutex;
use embassy_sync::signal::Signal;

static IRQ_SIGNAL: Signal<CriticalSectionRawMutex, ()> = Signal::new();
let radio_irq = interrupt::take!(SUBGHZ_RADIO);
radio_irq.set_handler(|_| {
    IRQ_SIGNAL.signal();
    unsafe { interrupt::SUBGHZ_RADIO::steal() }.disable();
});

```

这样, 在 `async fn main()` 中使用 `IRQ_SIGNAL.wait().await` 就可以随时等待中断信号量。

SubGhz 发送端

首先拼接报文, 这里直接手动拼接组合字节:

```

let mut payload = [0u8; 24];
let now = Instant::now();
let measurements = unwrap!(bmp280.measure(&mut delay));

payload[0] = b'M';

```

```

payload[1] = b'M';

payload[2..6].copy_from_slice(dev_addr.to_be_bytes().as_slice());
payload[6..14].copy_from_slice(now.as_millis().to_be_bytes().as_slice());
payload[14..18].copy_from_slice(measurements.temperature.to_be_bytes().as_slice());
payload[18..22].copy_from_slice(measurements.pressure.to_be_bytes().as_slice());
let checksum = payload[2..22]
    .iter()
    .fold(0u16, |acc, x| acc.wrapping_add(*x as u16));
info!("checksum: {:04x}", checksum);
payload[22..24].copy_from_slice(checksum.to_be_bytes().as_slice());

```

然后开始发送:

```

rfs.set_tx();
unwrap!(radio.set_irq_cfg(&CfgIrq::new().irq_enable_all(Irq::TxDone)));
unwrap!(radio.write_buffer(TX_BUF_OFFSET, &payload[..]));
unwrap!(radio.set_tx(Timeout::DISABLED));

radio_irq.enable();
IRQ_SIGNAL.wait().await;
rfs.set_off();

let (_, irq_status) = unwrap!(radio.irq_status());
if irq_status & Irq::TxDone.mask() != 0 {
    defmt::info!("TX done");
}
unwrap!(radio.clear_irq_status(irq_status));

```

总结起来发送过程需要如下步骤:

- 打开射频发送开关
- 设置中断, 开启 TxDone
- 写入数据 buffer
- 开始发送, 不使用 Timeout
- 开启中断
- 等待中断信号量
- 关闭射频开关
- 检查中断状态
- 清理中断状态

SubGhz 接收端

这里是接收端逻辑, `src/bin/subghz-bmp280-rx.rs`, 其中配置部分和发送端相同:


```

let mut buf = [0u8; 256];

rfs.set_rx();
unwrap!(radio.set_irq_cfg(
    &CfgIrq::new()
        .irq_enable_all(Irq::RxDone)
        .irq_enable_all(Irq::Timeout)
        .irq_enable_all(Irq::Err)
));
unwrap!(radio.read_buffer(RX_BUF_OFFSET, &mut buf));
unwrap!(radio.set_rx(Timeout::from_duration_sat(Duration::from_millis(500))));

radio_irq.unpend();
radio_irq.enable();

IRQ_SIGNAL.wait().await;
led_rx.set_low();
let (_, irq_status) = unwrap!(radio.irq_status());
unwrap!(radio.clear_irq_status(irq_status));

if irq_status & Irq::RxDone.mask() != 0 {
    let (_st, len, offset) = unwrap!(radio.rx_buffer_status());
    let packet_status = unwrap!(radio.lora_packet_status());
    let rssi = packet_status.rssi_pkt().to_integer();
    let snr = packet_status.snr_pkt().to_integer();
    info!(
        "RX done: rssi={}dBm snr={}dB len={} offset={}",
        rssi, snr, len, offset
    );
    let payload = &buf[offset as usize..offset as usize + len as usize];
    // Parse payload here
}

```

发送步骤如下:

- 打开射频接收开关
- 设置中断, 开启 RxDone, Timeout, Err
- 设置读入 buffer
- 开始接收, 这里使用 Timeout 5 秒
- 清理未处理中断状态, 否则会有观察到空中中断
- 开启中断
- 等待中断信号量
- 检查中断状态, 清理中断状态
- 通过 rx_buffer_status 获取 buffer 状态
- 通过 lora_packet_status 获取报文 rssi, snr 信息

运行结果

发送端上电之后, 每2秒采集一次传感器数据并发送.

接收端上电之后, 持续接收数据并同时打印在 defmt 调试和串口输出.

```
> cargo run --bin subghz-bmp280-rx --release
1.226162 INFO begin rx...
3.292868 INFO RX done: rssi=-42dBm snr=14dB len=24 offset=0
3.292969 DEBUG got BMP280 node raw=[0x4d, 0x4d, 0x72, 0x2e, 0x67, 0x28,
3.293173 INFO dev addr=722e6728 dev tick=22586 temp=21.633121'C pressure=
3.299479 INFO stats: Stats { status: Status { mode: 0k(StandbyRc), cmd:
3.299622 INFO begin rx...
```

串口输出, CSV 格式:

```
> picocom -b 115200 /dev/tty.usbmodem11203
addr=722e6728,rssi=-44,snr=14,temperature=16.304043,pressure=87621.96
addr=722e6728,rssi=-44,snr=14,temperature=16.306524,pressure=87621.96
addr=722e6728,rssi=-44,snr=13,temperature=16.309006,pressure=87621.83
addr=722e6728,rssi=-45,snr=13,temperature=16.311487,pressure=87621.81
addr=722e6728,rssi=-45,snr=13,temperature=16.313969,pressure=87621.66
```

总结

Rust Embassy 是一个非常好的嵌入式 Rust 开发框架, 通过它可以快速开发嵌入式应用. Rust Embassy 把 `async`, `await` 关键字带到了 Rust 嵌入式开发中, 其还有丰富的多任务支持, 多种同步元语支持. 通过它们, 我们可以很方便的开发多任务应用.

但它依然是一个很早期的框架, 还不够完善, 例如目前在 STM32WL 上缺乏 ADC 支持. 文档不够丰富, 部分库函数会随着开发进度有所变更, 给维护项目带来不小的困难.

在开发过程中, 往往能看到 `move` 语义, `ownership`, 类型系统等 Rust 的特性, 虽然这些特性在嵌入式开发中并不是必须的, 但是它们确实能带来更好的开发体验. 例如 `move/borrow` 保证对设备资源的唯一访问所有权. 通过类型安全的寄存器类型访问避免 C 语言中错误的寄存器访问, 经过 Rust 编译器优化后, 和 C 中的 `bit mask` 写法是等价的. 通过 "associated types" 保证设备和对应引脚的状态匹配.

Rust Embassy 隐藏了大部分嵌入式设备细节, 开发者不需要过多的关注设备初始化细节, 应用代码短小.

实际使用过程中, 也遇到了一些坑, 例如在写一个 PWM 例子时候, `embassy_time::Delay` 怎么都不工作, 添加了若干 debug 打印之后才发现, `embassy_time::Delay` 内部使用 `embassy_time::Instant` 实现, 默认情况下会使用 `TIM2`. 而选择的 PWM 输出 pin 正好是 `TIM2_CH2`, 两者互相干扰, 导致 `Delay` 不工作. 目前类型系统还不能保证 `Delay` 和 `Pwm`

不会使用同一个 `TIM` 设备. 最终的解决方法是使用 `cortex_m::delay::Delay`, 这是一个基于 SYSTICK 的实现.

本位未介绍 Embassy 的多任务功能, 在代码仓库里有一个简单的按钮控制闪灯频率的例子 `src/bin/button-control-blinky.rs`. 多任务的时候需要有 `.await` 调用让出时间片.

参考资料

- [Github: Embassy](#)
- [Embassy Documentation](#)
- [Embedded Rust & Embassy 系列教程](#)
- [STM32WLE5.pdf](#)

ALSO ON MY BLOG

Play with 2.13 inch E-Ink display 4 years ago · 3 comments 故事从买屏幕说起。	基于 Rust 的 K230 裸机嵌入式编程 - K230 ... 3 months ago · 2 comments 难度: 中等, 读者应具备嵌入式系统基础知识和 Rust 嵌入式开发基础	在 CircleCI 上使用 Rust(CircleCI Meets ... 8 years ago · 1 comment 最近由于频频遇到 travis-ci 的问题, 主要是 Linux 资源排队、macOS ...	为 ... 10 ... 本 ... 法 ...
---	---	--	--

What do you think?

5 Responses



Upvote



Funny



Love



Surprised



Angry



Sad

0 Comments

 Login ▼

G

Start the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS 

Name



Share

Best

Newest

Oldest

Be the first to comment.

Subscribe

Privacy

Do Not Sell My Data

猫·仁波切

猫·仁波切

andelf@gmail.com



andelf



bc12edd8ae2cb21

会研发的PM才是好OP.